AI PRACTICE - Flask Login/Register Example

This project is a simple Flask-based login and registration system using SQLite for storage. It demonstrates how to

What this project does

• Provides a registration form for new users.
• Stores users in an SQLite database (`data.sqlite3`).
• Hashes passwords before saving them using `werkzeug.security.generate_password_hash`.
• Validates login credentials using `werkzeug.security.check_password_hash`.
• Uses HTML templates for pages: home, login, register, and success.
• Provides a homepage button to download the generated PDF documentation.
• Logs important app behavior through the database and user interface.

What we did step by step

1. Created `flask_app.py` as the main Flask application.
2. Set up `templates/` with `index.html`, `login.html`, `register.html`, and `success.html`.
3. Added a SQLite database table in `init_db()` to store users.
4. Built `/register` and `/login` routes.
5. Used `generate_password_hash()` to store hashed passwords safely.
6. Updated login logic to verify passwords with `check_password_hash()`.
7. Converted existing plain-text passwords in `data.sqlite3` to hashed values.
8. Removed the temporary helper script used for conversion.
9. Added a homepage button and Flask route to download `project_documentation.pdf`.
10. Verified the app and database, ensuring all stored passwords are hashed.
11. Pushed the project to GitHub and resolved any merge conflicts.

Project structure

• `flask_app.py` — main application logic.
• `requirements.txt` — required Python packages.
• `data.sqlite3` — SQLite database file.
• `logi.txt` — optional plaintext append log from older compatibility code.
• `templates/` — HTML templates used by Flask.

How the programming works

`flask_app.py`

• Imports:
  - `Flask`, `render_template`, `request`, `send_from_directory` from Flask for web routing, templates, and serving fi
  - `os`, `sqlite3`, `datetime` from Python standard library.
  - `generate_password_hash`, `check_password_hash` from `werkzeug.security` for secure password handling.

• `DB_FILENAME`:
  - Sets the SQLite file path next to `flask_app.py` using `app.root_path`.

• `init_db()`:
  - Creates the `users` table if it does not exist.
  - The table has fields: `id`, `full_name`, `email`, `username`, `password`, `created_at`.
  - It ensures `email` is unique and `password` is stored as text.

• `@app.route('/')`:
  - Renders `index.html` as the home page.

• `@app.route('/login', methods=['GET', 'POST'])`:
  - GET: shows the login form.
  - POST: reads `email` and `password` from the form.
  - Queries the user row by email.
  - Verifies the provided password with `check_password_hash()`.
  - Shows `success.html` on success, otherwise returns the login form with an error.

• `@app.route('/register', methods=['GET', 'POST'])`:
  - GET: shows the registration form.
  - POST: reads `full_name`, `email`, and `password`.
  - Hashes the password with `generate_password_hash()`.
  - Inserts the user into the database with the current timestamp.
  - Shows `success.html` after successful registration.

• `@app.route('/download-documentation')`:
  - Serves the generated PDF file `project_documentation.pdf` from the project root.
  - Returns the file as an attachment so users can download it from the homepage.

• `if __name__ == '__main__':`
  - Calls `init_db()` again and runs the Flask development server with `debug=True`.

Key functions and libraries

Flask

• `Flask(__name__)` — create the app.
• `@app.route()` — define URL routes.
• `render_template()` — render an HTML template.
• `request.form.get()` — retrieve submitted form data.

SQLite (`sqlite3`)

• `sqlite3.connect(DB_FILENAME)` — open the database.
• `conn.cursor()` — create a cursor for SQL operations.
• `cursor.execute()` — run SQL statements.
• `cursor.fetchone()` — fetch a single query result.
• `conn.commit()` — save changes.
• `conn.close()` — close the connection.

Werkzeug security

• `generate_password_hash(password)` — hash a plain-text password.
• `check_password_hash(hashed_password, password)` — verify a password against its hash.

Templates

• The app uses HTML templates in `templates/` for each page.
• `register.html` collects full name, email, and password.
• `login.html` collects email and password.
• `success.html` shows a success message after login or registration.

Setup and running the project

1. Open a terminal in `C:\Users\TARIQ NASHEED\OneDrive\Desktop\AI PRACTICE`.
2. Create and activate a virtual environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

3. Install dependencies:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

4. Start the app:

```
.\.venv\Scripts\python.exe flask_app.py
```

5. Open `http://127.0.0.1:5000/` in your browser.

Security notes

• Passwords are hashed before storing in the database.
• Never store plain-text passwords in production.
• The app is for learning and practice only.
• For production, add HTTPS, input validation, stronger session management, and CSRF protection.

GitHub and repository notes

• This project has been pushed to a GitHub repository.
• Conflicts were resolved manually in `README.md`.
• `README.md` now contains complete documentation and project history.

PDF Documentation

A PDF version of this documentation is also generated in the project root as `project_documentation.pdf`.